PROBLEM 1 / 20 POINTS

Image warping and caricature

Einstein portrait | radial enlargement and a radius-dependent swirl

A. Forward radial warp

The grayscale image is 806 × 746. I used $(c_x, c_y) = (403, 373)$. For each input pixel (x, y), I computed $r = \text{hypot}(x - c_x, y - c_y)$ and $\theta = \text{atan2}(y - c_y, x - c_x)$. The new radius is $\sqrt{(\alpha r)}$. I converted back to Cartesian coordinates with cosine and sine, rounded to the nearest pixel, discarded out-of-bounds samples, and averaged intensities that landed on the same pixel.

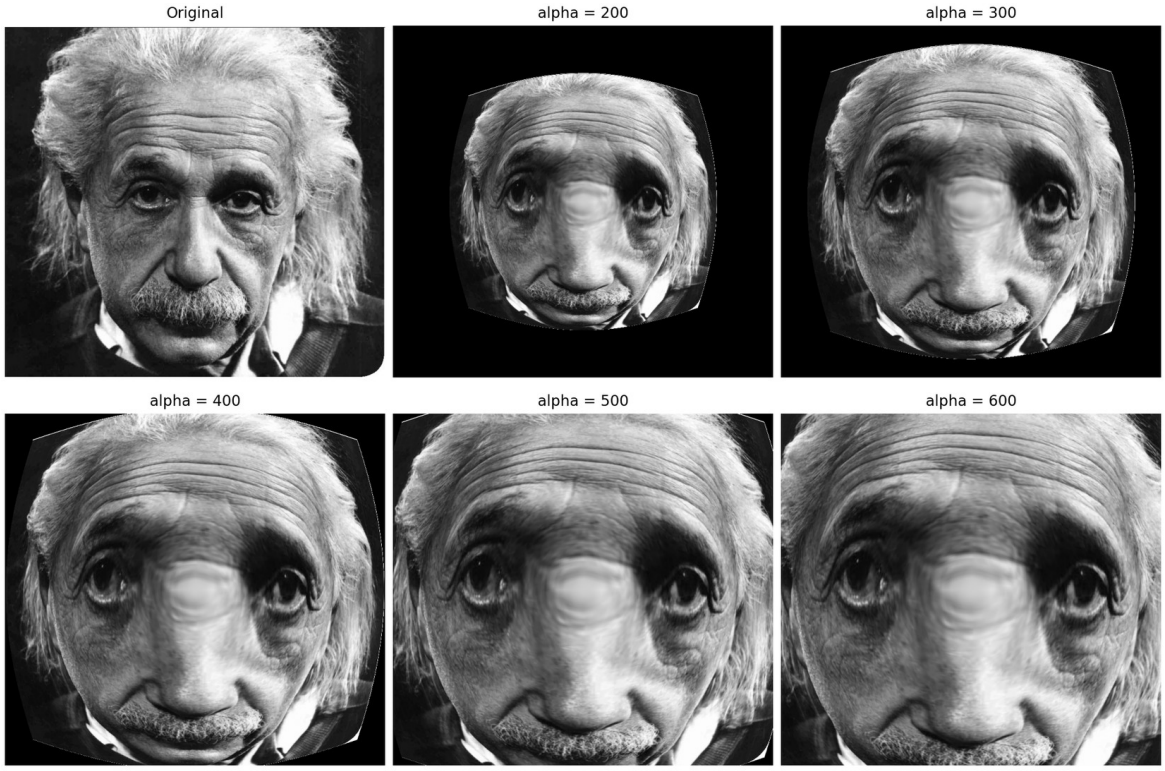


Figure 1. Alpha sweep using inverse sampling so that the shape can be compared without rounding gaps.

Preferred α = 600. The eyes and nose are enlarged, the face still reads clearly, and the warped image covers the output canvas. Lower values retain more of the outer portrait and leave more dark background at the frame edges.

B. Inverse mapping with bilinear interpolation

For each output pixel, I computed (r_o, θ_o) about the same center and sampled the input at $r = r_o^2/\alpha$ and $\theta = \theta_o$. For source coordinate $(i + a, j + b)$, the four neighbor weights are $(1 - a)(1 - b)$, $a(1 - b)$, $(1 - a)b$, and ab. Inverse sampling writes each valid destination once and eliminates 293,456 unassigned pixels from the rounded forward radial warp.

Swirl and interpolation

Forward scatter and inverse bilinear sampling shown side by side

C–D. Define and invert the swirl

I kept the radial map and added $\theta_o = \theta + (35\pi/180)(1 - r/R)$, where $R = \text{hypot}(c_x, c_y)$ is the maximum source radius. The angle changes most near the center and approaches the original angle near the corners. For the inverse, I first recovered $r = r_o^2/\alpha$, then subtracted $(35\pi/180)(1 - r/R)$ from θ_o and applied the same bilinear sampling.

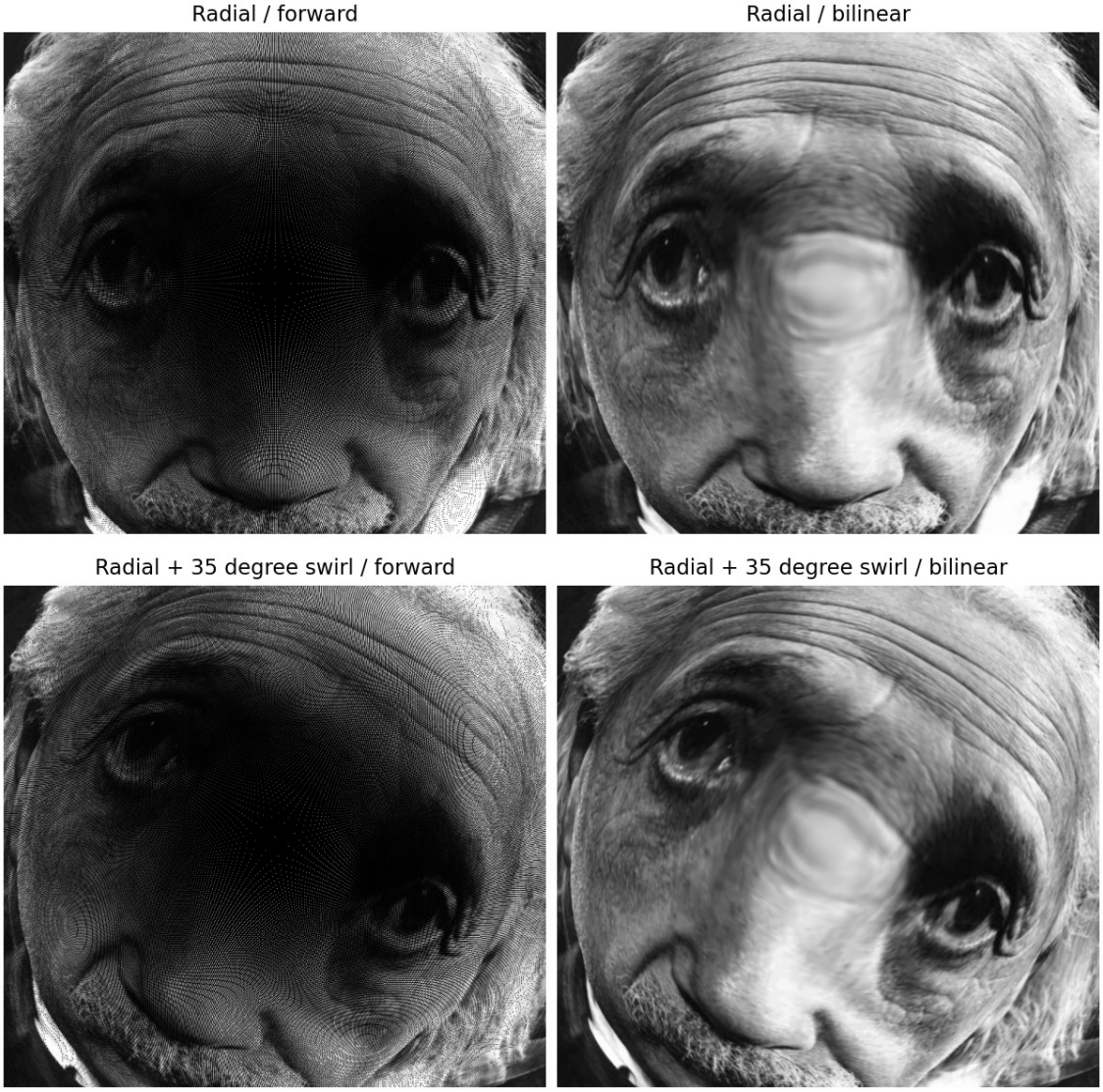


Figure 2. Row 1 shows the radial warp. Row 2 adds the swirl. At α = 600 the forward swirl leaves 298,147 unassigned pixels. Both inverse outputs cover the canvas.

Functions used. OpenCV imread. NumPy indices, hypot, arctan2, sqrt, cos, sin, rint, bincount and array indexing for manual bilinear interpolation. Matplotlib subplots, imshow and savefig.

PROBLEM 2 / 30 POINTS

Book matching and homography

Supplied frontal book photo and oblique scene | 640 × 480 pixels each

A–C. Inputs and SIFT detection

The assignment-supplied zen_book_frontal.png is the template and zen_book_perspective.png is the scene. SIFT found **593 template** and **783 scene** keypoints. Settings were nfeatures = 0 (no cap), nOctaveLayers = 3, contrastThreshold = 0.04, edgeThreshold = 10 and sigma = 1.6.

D. FLANN matching

A KD-tree index (algorithm 1, five trees) searched with 50 checks and k = 2. I kept the nearest match if its descriptor distance was below 0.70 times the second-nearest distance. Of 593 two-neighbor queries, 132 passed the ratio test.

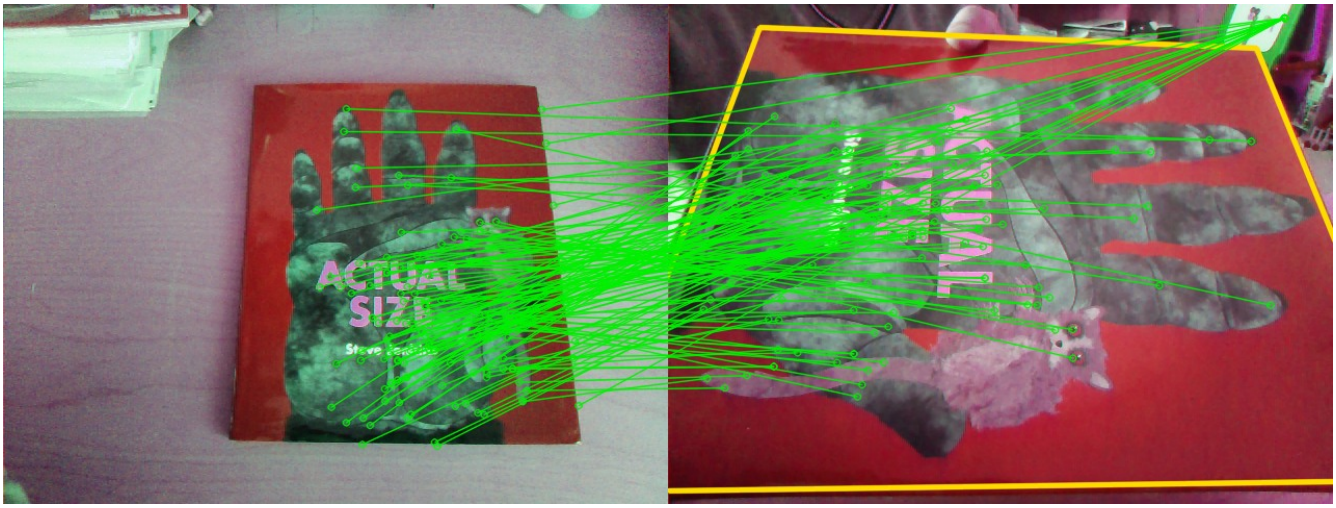


Figure 3. All 132 ratio-test matches, drawn in green.

E. Estimate the projective mapping

Of the 132 matches, 118 source points lay inside the book outline. RANSAC used a 3 px threshold, 5,000 iterations maximum and 0.999 confidence. It retained **113 inliers** with a forward reprojection RMSE of **1.48 px**. Pixel points are mapped by normalizing $H[x, y, 1]^T$ by its third component.

H | template to scene

-0.341476	-1.04421	589.239
0.643474	0.0104184	-124.905
-0.00111629	0.000296173	1

Projecting the four template cover corners gives (566.3, 42.0), (735.6, 457.1), (-79.0, 468.4), and (87.7, 22.4) px, in template corner order. The scene crops two of the corners. The yellow outline is clipped at the image boundary.

Validate both directions

Separate hand-marked targets check the projected landmark positions

E. Template to scene

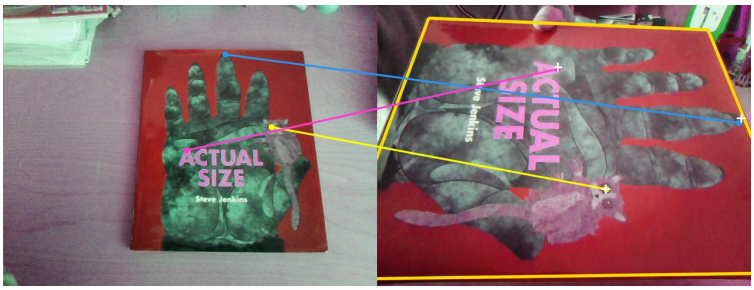


Figure 4. Colored segments connect each template landmark to its H projection. White crosses mark hand-picked scene locations.

Landmark	Input (px)	Projected (px)	Observed (px)	Error (px)
Animal eye	(459, 208)	(392.0, 314.3)	(391, 314)	1.05
Tall fingertip	(380, 84)	(618.9, 200.6)	(623, 193)	8.63
Letter A	(313, 248)	(308.5, 109.2)	(309, 106)	3.26

F. Scene to template

H inverse | scene to template

```
     0.075342        1.93668        197.507
    -0.800974       0.502605        534.743
   0.00032133     0.00201303         1.0621
```

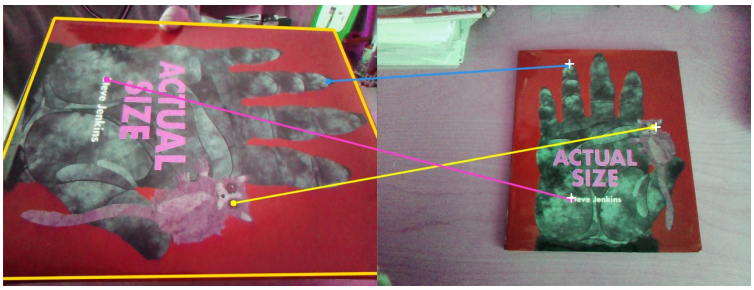


Figure 5. Three scene landmarks mapped back with H inverse. White crosses mark hand-picked template locations.

Landmark	Input (px)	Projected (px)	Observed (px)	Error (px)
Other animal eye	(394, 338)	(471.8, 208.1)	(477, 208)	5.23
Second fingertip	(556, 130)	(326.9, 103.0)	(329, 100)	3.65
Author initial	(177, 127)	(332.3, 332.3)	(332, 329)	3.32

The landmarks map to the expected cover details. Approximate hand placement and perspective make a fingertip less repeatable than the eye. Functions: SIFT_create, detectAndCompute, FlannBasedMatcher.knnMatch, findHomography, perspectiveTransform, drawMatches, polylines, and NumPy linalg.inv.

PROBLEM 3 / 20 POINTS

Epipolar geometry

IMG_4394.jpeg (wide) to IMG_4393.jpeg (close) | desk scene

A. SIFT, FLANN, and fundamental matrix

The two views share a desk, keyboard, and objects at different depths. Display regions were masked to reduce the influence of a highly textured plane. With the Problem 2 SIFT and FLANN settings and a 0.75 ratio threshold, the wide and close views produced 985 and 1,356 keypoints. Of 985 KNN queries, 104 passed the ratio test and 65 were geometric inliers. findFundamentalMat used USAC_MAGSAC, a 1.5 px threshold, 0.999 confidence, and up to 10,000 iterations.

F | normalized to unit Frobenius norm

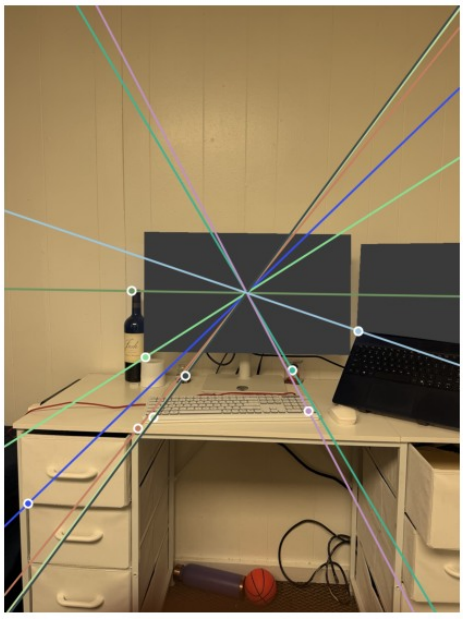
```
9.1322868e-07  8.26019379e-05  -0.0606376046
-8.30079211e-05  2.35708096e-06   0.0491443974
  0.059754843  -0.0517041196    0.993812829
```

B–C. Rank and epipolar constraint

The singular values of F are 0.99998098, 0.00616783, and 1.09e-18. The third is numerically zero, giving rank 2. The wide-view point x1 = (377.077789, 1042.078125, 1) corresponds to x2 = (134.410339, 1324.780151, 1) in the close view. Using the unrounded values, $x2^T$ F x1 = 0.00532233. The Sampson error is 0.07534 px for this pair, with an inlier median of 0.31468 px. The small residual is consistent with feature localization and fitting error.

D. Corresponding epipolar lines

IMG_4394.jpeg



IMG_4393.jpeg

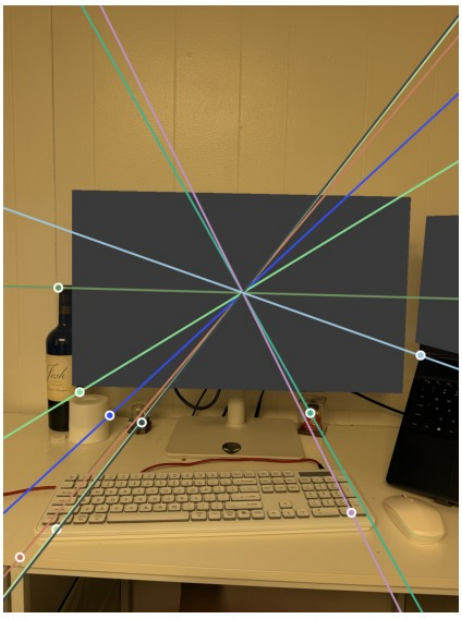


Figure 6. Nine matched points and their corresponding lines in both images. The matching line and point use the same color. Display regions are shaded.

The line in view 2 is F x_1 and the line in view 1 is $F^T x_2$. Functions: computeCorrespondEpilines, clipLine, line, circle and NumPy linalg.svd and matrix_rank.

PROBLEM 4 / 50 POINTS

Calibration and camera pose

Photo A IMG_4396.jpeg | Photo B IMG_4398.jpeg

A. Approximate intrinsic calibration

Photo A is the near-overhead image. The cover is approximately 4.00 × 5.50 inches, excluding the strap, and the lens-to-cover distance is approximately 7.00 inches. Averaging the opposite annotated edges gives d_x = 727.19 px and d_y = 920.18 px. Similar triangles give $f_x = d_x d_Z / d_X$ and $f_y = d_y d_Z / d_Y$. I set zero skew and used the center (576, 768) of the 1152 × 1536 image as the principal point.

K | focal lengths and principal point in pixels

1272.58	0	576
0	1171.14	768
0	0	1

The two focal estimates differ by about 8% relative to their mean. The approximate physical dimensions, edge picks, slight tilt in Photo A, and ignored distortion limit the calibration. The focal estimates also scale directly with the assumed distance.

B. Pose from the cover plane

I used inches and put the world origin at the cover center, with X right, Y down and Z into the notebook. For Photo A I assumed a fronto-parallel plane, allowed an in-plane roll from the cover edges, and back-projected the center to depth 7 in to get t_A.

SIFT found 160 and 151 points within the respective cover masks, with the raised strap excluded. FLANN and a 0.80 ratio yielded 35 matches. USAC_MAGSAC (5 px, 10,000 iterations, 0.999 confidence) kept **15 inliers** with **2.02 px** homography RMSE.

H | Photo A to Photo B

0.739517	-0.261143	238.235
0.00936739	0.429296	216.679
-4.36443e-05	-0.00030852	1

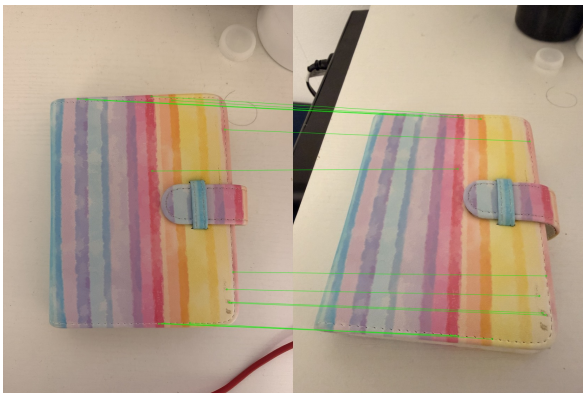


Figure 7. Homography inliers restricted to the notebook cover in both photos.

Projection and camera center

Recover a rigid camera from the plane mapping and test the overlay

For Z = 0 the plane projection in Photo A is $G_A = K[r_{A1}\ r_{A2}\ t_A]$. Multiplying by H gives G_B up to scale. I computed $K^{-1}G_B$, scaled its first two columns to have unit mean norm, chose the sign that places the cover in front of the camera, and formed the third column by cross product. An SVD projected this estimate onto the nearest proper rotation. Finally I formed $P_B = K[R \mid t]$.

P_A | Photo A projection

1272.361	23.77188	576	3839.5
-21.87691	1170.936	768	5734.75
0	0	1	7

P_B | Photo B projection

1229.869	-370.5968	548.9314	3934.224
7.454766	660.3775	1235.006	5248.306
-0.06650031	-0.4761314	0.8768561	6.619035

I factored the left 3 × 3 block by RQ, chose a positive diagonal in K and recovered t from $Kt = P_B[:, 3]$. The recovered K agrees with the input K to 4.55×10^{-13}. The camera center is **$C = -R^T t$ = (0.338, 3.035, -5.875) inches**. Substituting $[C, 1]^T$ in P_B gives zero to floating-point precision.

The rigid pose projects the 15 planar matches with a 4.39 px RMSE. This is higher than the free homography's 2.02 px because the pose constrains the rotation and retains the approximate calibration. The cover-edge corner annotations differ from the homography projections by 10.08 px RMSE.



Figure 8. The same 2 in square projected into Photos A and B (left). The cube in Photo B (right) has a 2 in base and rises 2 in along negative Z.

Functions: findHomography, perspectiveTransform, drawMatches, polylines, NumPy linalg.solve and linalg.svd, and SciPy linalg.rq.

Augmented reality

Pygame and OpenGL | original Wavefront OBJ and diffuse materials

C. Render the colored OBJ into Photo B

The supplied viewer imports an OBJ loader that was absent from the ZIP. ObjLoader.py reads the supplied vertex and face data and diffuse colors from the MTL file. I used it with ObjViz and captured the model in an OpenGL window.

The AR script loads Photo B as the background, sets the projection matrix from K, and loads the recovered pose after flipping the camera Y and Z axes to OpenGL's convention. The OBJ sits on the cover plane and rises along negative world Z. The depth buffer resolves visibility between faces. The OpenGL and pinhole vertex projections agree within 6.82×10^{-13} px.

MeshLab 2020.09 aborted while importing the supplied OBJ. I imported a PLY copy with the same vertices and face colors, splitting each quad into two triangles. ObjViz and the AR renderer use the original OBJ and MTL.

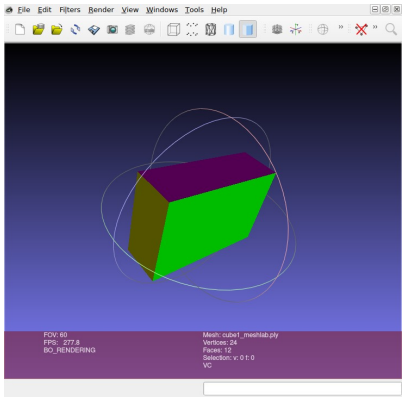


Figure 9. The colored cube viewed in MeshLab from the PLY copy.

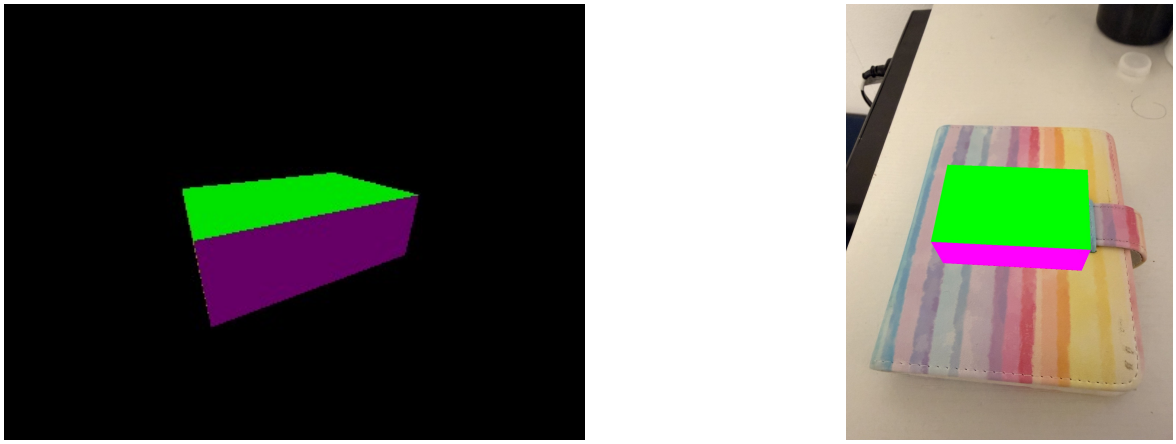


Figure 10. ObjViz preview (left) and calibrated OBJ composite (right). The original material colors are read from cube1_color.mtl.

Functions used. Pygame display.set_mode, display.flip and event.wait. PyOpenGL glDrawPixels, glLoadMatrixd, glVertex3f and glReadPixels. Pillow Image.open and Image.frombytes.